Lists: Python's Dynamic Workhorse

Week 2 – Monday Lecture

Course: Programming for AI
Dr. Prateek, AP
School of Artificial Intelligence

MTech AI

Today's Agenda (1 Hour)

1. Recap: Week 1 in 60 Seconds *5 min*
2. Defining the "Workhorse": What is a List? *15 min*
3. Checking for Membership: `in` / `not in` *10 min*
4. Programmatic Growth: `.append()`, `.extend()`, `.insert()` *25 min*
5. Practice & Wrap-up *5 min*

Key Takeaway

So far your programs could store **one** value per box. Today, one box learns to hold **many** values – and to grow.

Recap: Week 1 in 60 Seconds

Data & Memory

- Variables are labelled boxes: `int`, `float`, `str`, `bool`.
- `input()` reads from the user; `print()` writes to the screen.

Decision Control

- Comparisons (==, >, ...) always return a Boolean.
- `if` / `elif` / `else` let a program choose *one* path based on that Boolean.

```
marks = int(input('Enter marks: '))
if marks >= 60:
    print('Pass')
else:
    print('Fail')
```

Think About It

Everything so far stored **one** value at a time. What if you needed to store the marks of **30** students?

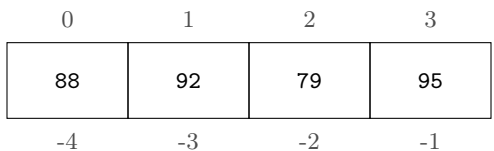
Defining the "Workhorse": What is a List?

A **list** is an **ordered**, **mutable** collection of objects.

- **Ordered** – items keep the position you placed them in, and each has an index (starting at 0).
- **Mutable** – you can change, add, or remove items after creation.
- **Dynamic** – unlike arrays in languages such as C or Java, a Python list grows and shrinks on demand; you never declare a fixed size.
- **Heterogeneous** – a single list can mix data types, even other lists.

```
scores = [88, 92, 79, 95]
mixed  = ['AI', 3.14, True, [1, 2]]    # perfectly legal
```

Indexing: Reading Inside the Box



`scores[0]` → 88 `scores[-1]` → 95 `scores[2]` → 79

Positive indices count from the left starting at `0`; negative indices count from the right starting at `-1`.

Checking for Membership: `in` / `not in`

- To check whether a value exists inside a list, Python offers the "super" operators `in` and `not in`.
- Both return a **Boolean** – and read almost like English.
- Far more readable (and less error-prone) than writing a manual loop to search item by item.

```
fruits = ['apple', 'banana', 'mango']

print('banana' in fruits)        # True
print('grape' in fruits)         # False
print('grape' not in fruits)     # True
```

Predict the Output

```
enrolled = ['Asha', 'Raj', 'Meera']
name = 'Kabir'
if name not in enrolled:
    print(name, 'must register first.')
```

What gets printed? Trace it before checking.

Programmatic Growth: `.append()`

- `.append(item)` adds a **single** object to the **end** of the list.
- The list grows by exactly one element – even if that element is itself a list.

```
queue = ['Asha', 'Raj']
queue.append('Meera')
print(queue)                # ['Asha', 'Raj', 'Meera']

queue.append(['Kabir', 'Dev'])
print(queue)                # ['Asha', 'Raj', 'Meera', ['Kabir', 'Dev']]
print(len(queue))           # 4  -- not 5!
```

Common Mistake

`.append()` never "unpacks" what you give it. Appending a list adds **one** new item, which happens to be a list – it does not merge the items individually.

Programmatic Growth: `.extend()`

- `.extend(other_list)` takes a second list and merges **each of its items** into the first, one by one.
- Use `.extend()` when you want to combine two lists into one flat list.

```
team_a = ['Asha', 'Raj']
team_b = ['Kabir', 'Dev']

team_a.extend(team_b)
print(team_a)                # ['Asha', 'Raj', 'Kabir', 'Dev']
print(len(team_a))           # 4
```

Think About It

Compare this with the previous slide: `queue.append(['Kabir', 'Dev'])` gave a list of length 4 with a nested list inside. What would `queue.extend(['Kabir', 'Dev'])` have given instead?

Programmatic Growth: `.insert(index, value)`

- `.insert(index, value)` places an object at a **specific position**, pushing everything after it one step to the right.
- This is computationally **expensive** on long lists – every following element must physically shift to make room.
- Compare with `.append()`, which only touches the end of the list and is much cheaper.

```
queue = ['Asha', 'Raj', 'Meera']
queue.insert(1, 'Priya')
print(queue)                # ['Asha', 'Priya', 'Raj', 'Meera']
```

Key Takeaway

Choose the right tool: `.append()` for the common case (adding to the end), `.extend()` for merging two lists, and `.insert()` only when position genuinely matters.

Worked Example: Building a Attendance List

```
attendance = []                          # start empty

attendance.append('Asha')
attendance.append('Raj')
attendance.extend(['Kabir', 'Dev'])
attendance.insert(0, 'Meera')            # Meera arrived first!

print(attendance)
print('Raj' in attendance)
print(len(attendance))
```

Think About It

Trace this by hand, line by line. What does `attendance` look like after each call? What is the final `len(attendance)`?

Practice (5 minutes)

Exercise

Starting from `cart = ['bread', 'milk']`, write the calls needed to:

1. Add `'eggs'` to the end.
2. Merge in the list `['butter', 'jam']` so both items appear individually.
3. Insert `'coupon'` at the very front of the cart.
4. Check whether `'milk'` is still in the cart.

Summary: Key Takeaways

- A **list** is ordered, mutable, dynamic, and can hold mixed data types.
- `in` / `not in` test membership and return a Boolean.
- `.append()` adds one item to the end.
- `.extend()` merges another list's items in, one by one.
- `.insert(index, value)` places an item at a specific position, at a higher computational cost.

Key Takeaway

Lists let a single variable represent an entire, growing collection of data – the foundation for storing datasets an AI model will later learn from.

Looking Ahead: Friday

- **Next session (Hour 2):** Advanced slicing – extracting fragments of a list using `[start:stop:step]`.
- **Before Friday:** Be comfortable with list indexing (positive and negative) from today's slides.

Questions? Thank you!